

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Operating Systems COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks:25

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

I, **Ashira Bridget Susan S (192524316)**, (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



S.No	Comparative Analysis Questions	Marks
	Compare Contiguous and Linked file allocation methods in terms of access speed, memory utilization, and fragmentation . Identify which method is more suitable for sequential file access and justify your answer.	5
2	Differentiate between Linked and Indexed file allocation with respect to random access capability, storage overhead, and reliability . Explain which method is more efficient for large files .	5
3	Compare FCFS and SSTF disk scheduling algorithms based on seek time, fairness, and the possibility of starvation . Discuss which algorithm performs better under heavy disk load .	5
4	Analyze the differences between SCAN and C-SCAN disk scheduling algorithms in terms of head movement, response time, and uniformity of service . Which algorithm is more suitable for real-time systems and why?	5
5	Compare LOOK and C-LOOK disk scheduling algorithms with respect to efficiency, total head movement, and practical implementation . Explain which algorithm provides better performance and under what conditions.	5
6	Compare Contiguous, Linked, and Indexed file allocation methods in terms of file growth, block allocation overhead, direct access, and fragmentation . A multimedia server stores files whose sizes frequently increase after creation. Evaluate which allocation method would be most suitable and justify your choice based on performance and scalability .	5
7	Compare Indexed and Contiguous allocation for a database system that performs frequent random reads and writes on large files . Analyze their impact on access time, metadata overhead, disk-space utilization, and file expansion . Determine which method would provide better overall performance and justify your answer.	5
8	A disk-intensive server receives hundreds of I/O requests simultaneously, with requests distributed across the entire disk. Compare SSTF, SCAN, and LOOK in terms of average seek time, starvation, fairness, and throughput . Evaluate which algorithm would provide the best balance between performance and fairness under sustained heavy workload.	5
9	A real-time monitoring system generates disk requests continuously from multiple processes. Compare FCFS, SCAN, C-SCAN, and SSTF based on response-time predictability, starvation risk, fairness, and head movement . Identify the most appropriate algorithm for the system and justify why predictable response time is more important than minimum average seek time .	5
10	A large scientific-data server performs both sequential and random access to files stored on an HDD. Compare Linked allocation and Indexed allocation together with SSTF and LOOK	

1. Compare Contiguous and Linked file allocation methods in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.

Answer:

Feature	Contiguous Allocation	Linked Allocation
Access Speed	Very fast (blocks are stored together)	Slower (must follow links between blocks)
Memory Utilization	May waste space due to external fragmentation	Better utilization since blocks can be anywhere
Fragmentation	Suffers from external fragmentation	No external fragmentation
File Growth	Difficult (needs continuous free space)	Easy (new blocks can be added anywhere)
Random Access	Easy	Difficult

Conclusion:

For **sequential file access**, **Contiguous Allocation** is more suitable because all file blocks are stored next to each other. This minimizes disk head movement and provides faster reading, making it ideal for multimedia files and programs that read data sequentially.

2. Differentiate between Linked and Indexed file allocation with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.

Answer:

Feature	Linked Allocation	Indexed Allocation
Random Access	Poor (must traverse links)	Excellent (index gives direct block addresses)
Storage Overhead	Low (one pointer per block)	Higher (requires an index block)
Reliability	Pointer corruption can lose remaining file	Only index block is critical
File Expansion	Easy	Easy
Access Speed	Slower	Faster

Conclusion:

Indexed Allocation is more efficient for **large files** because it supports direct/random access. Although it uses additional space for the index block, it significantly improves performance when accessing data from different locations in the file.

3. Compare FCFS and SSTF disk scheduling algorithms based on seek time, fairness, and the possibility of starvation. Discuss which algorithm performs better under heavy disk load.

Answer:

Feature	FCFS	SSTF
Seek Time	Higher	Lower
Fairness	High	Lower
Starvation	No starvation	Starvation may occur
Implementation	Simple	Slightly complex
Performance	Moderate	Better

Conclusion:

Under **heavy disk load**, **SSTF (Shortest Seek Time First)** performs better because it always services the nearest request, reducing average seek time. However, requests far from the current head position may wait for a long time, leading to starvation.

4. Analyze the differences between SCAN and C-SCAN disk scheduling algorithms in terms of head movement, response time, and uniformity of service. Which algorithm is more suitable for real-time systems and why?

Answer:

Feature	SCAN	C-SCAN
Head Movement	Moves in both directions	Moves in one direction only
Response Time	Varies depending on request position	More uniform response time
Uniformity of Service	Moderate	High
Fairness	Good	Better
Starvation	No	No

Conclusion:

C-SCAN is more suitable for **real-time systems** because it provides a more uniform waiting time for all requests. Every request waits for approximately one complete scan cycle, making response time more predictable.

5. Compare LOOK and C-LOOK disk scheduling algorithms with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.

Answer:

Feature	LOOK	C-LOOK
Head Movement	Moves in both directions	Moves in one direction only
Total Head Movement	Lower	Slightly higher due to jump
Efficiency	High	High
Fairness	Good	Better
Practical Use	General-purpose systems	Systems requiring uniform response time

Conclusion:

- **LOOK** provides better performance when minimizing total head movement is the priority because it reverses direction only after servicing the last request.
- **C-LOOK** performs better when fairness and uniform response time are more important, such as in systems handling many simultaneous requests.

Overall: LOOK is preferred for maximum efficiency, while C-LOOK is preferred for predictable service.

6. Compare Contiguous, Linked, and Indexed file allocation methods in terms of file growth, block allocation overhead, direct access, and fragmentation. A multimedia server stores files whose sizes frequently increase after creation. Evaluate which allocation method would be most suitable and justify your choice based on performance and scalability.

Answer:

Feature	Contiguous Allocation	Linked Allocation	Indexed Allocation
File Growth	Difficult	Easy	Easy
Block Allocation Overhead	Low	Pointer in every block	Index block required
Direct Access	Yes	No	Yes
Fragmentation	External fragmentation	No external fragmentation	No external fragmentation
Scalability	Poor	Good	Excellent

Conclusion:

For a multimedia server where file sizes frequently increase, **Indexed Allocation** is the best choice. It allows files to grow easily without requiring contiguous space, supports direct access for fast retrieval, avoids external fragmentation, and scales well for large multimedia files.

7. Compare Indexed and Contiguous allocation for a database system that performs frequent random reads and writes on large files. Analyze their impact on access time, metadata overhead, disk-space utilization, and file expansion. Determine which method would provide better overall performance and justify your answer.

Answer:

Feature	Contiguous Allocation	Indexed Allocation
Access Time	Very fast	Fast
Metadata Overhead	Very low	Higher (index block)
Disk Space Utilization	May waste space due to fragmentation	Better utilization
File Expansion	Difficult	Easy
Random Read/Write	Good initially	Excellent

Conclusion:

For a database system performing frequent random reads and writes on large files, **Indexed Allocation** provides better overall performance. Although it requires extra storage for the index block, it offers efficient random access, easy file expansion, and better disk-space utilization.

8. A disk-intensive server receives hundreds of I/O requests simultaneously, with requests distributed across the entire disk. Compare SSTF, SCAN, and LOOK in terms of average seek time, starvation, fairness, and throughput. Evaluate which algorithm would provide the best balance between performance and fairness under sustained heavy workload.

Answer:

Feature	SSTF	SCAN	LOOK
Average Seek Time	Lowest	Low	Lowest among SCAN family
Starvation	Possible	No	No
Fairness	Moderate	Good	Good
Throughput	High	High	Very High

Conclusion:

For sustained heavy workloads, **LOOK** provides the best balance between performance and fairness. It reduces unnecessary head movement by stopping at the last request instead of the disk end, resulting in lower seek time than SCAN while avoiding the starvation problem of SSTF.

9. A real-time monitoring system generates disk requests continuously from multiple processes. Compare FCFS, SCAN, C-SCAN, and SSTF based on response-time predictability, starvation risk, fairness, and head movement. Identify the most appropriate algorithm for the system and justify why predictable response time is more important than minimum average seek time.

Answer:

Feature	FCFS	SCAN	C-SCAN	SSTF
Response-Time Predictability	Moderate	Good	Excellent	Poor
Starvation Risk	None	None	None	Possible
Fairness	High	Good	Very High	Low
Head Movement	High	Moderate	Moderate	Lowest

Conclusion:

C-SCAN is the most appropriate algorithm for a real-time monitoring system because it provides predictable response times and equal waiting time for all requests. In real-time systems, consistent response is more important than achieving the minimum average seek time, since delayed responses may affect system reliability.

10. A large scientific-data server performs both sequential and random access to files stored on an HDD. Compare Linked allocation and Indexed allocation together with SSTF and LOOK disk scheduling as a combined file-management strategy. Analyze how the choice of file allocation affects disk access patterns and how the scheduling influences overall performance. Recommend the best combination for minimizing access delay while maintaining reliable file access, with suitable justification.

Answer:

Feature	Linked Allocation	Indexed Allocation
Sequential Access	Good	Good
Random Access	Poor	Excellent
Reliability	Lower (pointer failure affects file)	Higher
File Expansion	Easy	Easy
Overall Performance	Moderate	High
Scheduling Algorithm	SSTF	LOOK
Seek Time	Lowest	Low
Starvation	Possible	No
Fairness	Moderate	Good
Overall Reliability	Moderate	High

Conclusion:

The best combination is **Indexed Allocation + LOOK Disk Scheduling**.

- **Indexed Allocation** provides efficient sequential and random access, supports easy file expansion, and offers better reliability.
- **LOOK** minimizes unnecessary head movement while avoiding starvation, providing good throughput and fairness.

Therefore, **Indexed Allocation with LOOK** minimizes access delay, maintains reliable file access, and delivers the best overall performance for large scientific-data servers.